

создай example1.jmx файл для jMeter в котором выполняется нагрузочное тестирование UI интерфейса вот этой формы <https://somovstudio.github.io/test.html>

 Thought for 14 seconds >

Вот готовый файл example1.jmx для нагрузочного тестирования страницы авторизации. Скрипт эмулирует действия реального пользователя: открывает страницу, вводит логин и пароль, нажимает кнопку «Войти» и проверяет результат.

Содержимое файла example1.jmx

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<jmeterTestPlan version="1.2" properties="5.0" jmeter="5.6.3">
  <hashTree>
    <TestPlan guiclass="TestPlanGui" testclass="TestPlan" testname="Нагрузочное те
```

```

    стирование формы авторизации" enabled="true">
      <stringProp name="TestPlan.comments">Тестирование UI формы https://somovstud
io.github.io/test.html</stringProp>
      <boolProp name="TestPlan.functional_mode">>false</boolProp>
      <boolProp name="TestPlan.tearDown_on_shutdown">>true</boolProp>
      <boolProp name="TestPlan.serialize_threadgroups">>false</boolProp>
      <elementProp name="TestPlan.user_defined_variables" elementType="Arguments"
guiclass="ArgumentsPanel" testclass="Arguments" testname="Пользовательские перемен
ные" enabled="true">
        <collectionProp name="Arguments.arguments">
          <elementProp name="BASE_URL" elementType="Argument">
            <stringProp name="Argument.name">BASE_URL</stringProp>
            <stringProp name="Argument.value">https://somovstudio.github.io</strin
gProp>
            <stringProp name="Argument.metadata">=</stringProp>
          </elementProp>
        </collectionProp>
      </elementProp>
    </TestPlan>
    <hashTree>
      <!-- Группа потоков: 50 пользователей, 10 циклов, разогрев 5 секунд -->
      <ThreadGroup guiclass="ThreadGroupGui" testclass="ThreadGroup" testname="Гру
ппа потоков" enabled="true">
        <stringProp name="ThreadGroup.on_sample_error">continue</stringProp>
        <elementProp name="ThreadGroup.main_controller" elementType="LoopControlle

```

```

r" guiclass="LoopControlPanel" testclass="LoopController" testname="Циклический ко
нтроллер" enabled="true">
    <boolProp name="LoopController.continue_forever">false</boolProp>
    <intProp name="LoopController.loops">10</intProp>
</elementProp>
<stringProp name="ThreadGroup.num_threads">50</stringProp>
<stringProp name="ThreadGroup.ramp_time">5</stringProp>
<boolProp name="ThreadGroup.scheduler">false</boolProp>
<stringProp name="ThreadGroup.duration"></stringProp>
<stringProp name="ThreadGroup.delay"></stringProp>
<boolProp name="ThreadGroup.same_user_on_next_iteration">true</boolProp>
</ThreadGroup>
<hashTree>
    <!-- Постоянный таймер – задержка между шагами внутри одного пользователя
-->
    <ConstantTimer guiclass="ConstantTimerGui" testclass="ConstantTimer" testn
ame="Постоянный таймер" enabled="true">
        <stringProp name="ConstantTimer.delay">1000</stringProp>
        <stringProp name="ConstantTimer.staticDelay">true</stringProp>
    </ConstantTimer>
</hashTree/>

    <!-- HTTP Cookie Manager – для управления сессией -->
    <CookieManager guiclass="CookiePanel" testclass="CookieManager" testname
="Менеджер Cookie" enabled="true">

```

```
<collectionProp name="CookieManager.cookies"/>
<boolProp name="CookieManager.clearEachIteration">true</boolProp>
<boolProp name="CookieManager.controlledByThreadGroup">false</boolProp>
</CookieManager>
<hashTree/>

<!-- HTTP Request Defaults – базовые настройки запросов -->
<ConfigTestElement guiclass="HttpDefaultsGui" testclass="ConfigTestElement"
  testname="HTTP Request Defaults" enabled="true">
  <elementProp name="HTTPSampler.Arguments" elementType="Arguments" guiclass="HTTPArgumentsPanel"
    testclass="Arguments" testname="Параметры запроса" enabled="true">
    <collectionProp name="Arguments.arguments"/>
  </elementProp>
  <stringProp name="HTTPSampler.domain">somovstudio.github.io</stringProp>
  <stringProp name="HTTPSampler.port"></stringProp>
  <stringProp name="HTTPSampler.protocol">https</stringProp>
  <stringProp name="HTTPSampler.contentEncoding"></stringProp>
  <stringProp name="HTTPSampler.path"></stringProp>
  <stringProp name="HTTPSampler.method">GET</stringProp>
  <boolProp name="HTTPSampler.follow_redirects">true</boolProp>
  <boolProp name="HTTPSampler.auto_redirects">false</boolProp>
  <boolProp name="HTTPSampler.use_keepalive">true</boolProp>
  <boolProp name="HTTPSampler.DO_MULTIPART_POST">false</boolProp>
  <boolProp name="HTTPSampler.BROWSER_COMPATIBLE_MULTIPART">false</boolProp>
```

p>

```
<stringProp name="HTTPSampler.implementation">HttpClient4</stringProp>
</ConfigTestElement>
<hashTree/>
```

```
<!-- 1 Шаг: Открыть страницу (GET /test.html) -->
```

```
<HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProx
y" testname="01 – Открыть страницу авторизации" enabled="true">
```

```
<boolProp name="HTTPSampler.postBodyRaw">false</boolProp>
```

```
<elementProp name="HTTPSampler.Arguments" elementType="Arguments" guicla
ss="HTTPArgumentsPanel" testclass="Arguments" testname="Параметры запроса" enabled
="true">
```

```
<collectionProp name="Arguments.arguments"/>
```

```
</elementProp>
```

```
<stringProp name="HTTPSampler.domain"></stringProp>
```

```
<stringProp name="HTTPSampler.port"></stringProp>
```

```
<stringProp name="HTTPSampler.protocol"></stringProp>
```

```
<stringProp name="HTTPSampler.contentEncoding"></stringProp>
```

```
<stringProp name="HTTPSampler.path">/test.html</stringProp>
```

```
<stringProp name="HTTPSampler.method">GET</stringProp>
```

```
<boolProp name="HTTPSampler.follow_redirects">true</boolProp>
```

```
<boolProp name="HTTPSampler.auto_redirects">false</boolProp>
```

```
<boolProp name="HTTPSampler.use_keepalive">true</boolProp>
```

```
<boolProp name="HTTPSampler.DO_MULTIPART_POST">false</boolProp>
```

```
<boolProp name="HTTPSampler.BROWSER_COMPATIBLE_MULTIPART">false</boolPro
```

p>

```
<stringProp name="HTTPSampler.implementation">HttpClient4</stringProp>
</HTTPSamplerProxy>
<hashTree>
  <!-- Проверка: страница загрузилась (заголовок "Авторизация") -->
  <ResponseAssertion guiclass="AssertionGui" testclass="ResponseAssertion"
testname="Проверка: страница содержит заголовок 'Авторизация'" enabled="true">
    <collectionProp name="Asserion.test_strings">
      <stringProp name="0">&lt;h2&gt;Авторизация&lt;/h2&gt;</stringProp>
    </collectionProp>
    <stringProp name="Assertion.custom_message">Страница не содержит загол
овок Авторизация</stringProp>
    <stringProp name="Assertion.test_field">Assertion.response_data</strin
gProp>
    <boolProp name="Assertion.assume_success">false</boolProp>
    <intProp name="Assertion.assertion_scope">2</intProp>
  </ResponseAssertion>
  <hashTree/>
</hashTree>
```

<!-- **2** Шаг: Ввести логин и пароль (имитация через GET-параметры, т.к. форма не отправляет данные на сервер) -->

<!--

ВАЖНО: Данная форма обрабатывает ввод на стороне клиента (JavaScript) и НЕ отправляет

данные на сервер. Поэтому реальный POST-запрос отсутствует.

Для нагрузочного тестирования мы имитируем "ввод" как отдельный GET-запрос с параметрами,

чтобы проверить, что сервер отдаёт страницу с формой даже при передаче параметров.

-->

```
<HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy"
testname="02 – Ввод логина и пароля (имитация)" enabled="true">
  <boolProp name="HTTPSampler.postBodyRaw">false</boolProp>
  <elementProp name="HTTPSampler.Arguments" elementType="Arguments" guiclass="HTTPArgumentsPanel"
testclass="Arguments" testname="Параметры запроса" enabled="true">
    <collectionProp name="Arguments.arguments">
      <elementProp name="login" elementType="HTTPArgument">
        <boolProp name="HTTPArgument.always_encode">false</boolProp>
        <stringProp name="Argument.value">admin</stringProp>
        <stringProp name="Argument.metadata">=</stringProp>
        <boolProp name="HTTPArgument.use_equals">true</boolProp>
        <stringProp name="Argument.name">login</stringProp>
      </elementProp>
      <elementProp name="pass" elementType="HTTPArgument">
        <boolProp name="HTTPArgument.always_encode">false</boolProp>
        <stringProp name="Argument.value">0000</stringProp>
        <stringProp name="Argument.metadata">=</stringProp>
        <boolProp name="HTTPArgument.use_equals">true</boolProp>
      </elementProp>
    </collectionProp>
  </elementProp>
</HTTPSamplerProxy>
```

```

        <stringProp name="Argument.name">pass</stringProp>
      </elementProp>
    </collectionProp>
  </elementProp>
  <stringProp name="HTTPSampler.domain"></stringProp>
  <stringProp name="HTTPSampler.port"></stringProp>
  <stringProp name="HTTPSampler.protocol"></stringProp>
  <stringProp name="HTTPSampler.contentEncoding"></stringProp>
  <stringProp name="HTTPSampler.path">/test.html</stringProp>
  <stringProp name="HTTPSampler.method">GET</stringProp>
  <boolProp name="HTTPSampler.follow_redirects">true</boolProp>
  <boolProp name="HTTPSampler.auto_redirects">false</boolProp>
  <boolProp name="HTTPSampler.use_keepalive">true</boolProp>
  <boolProp name="HTTPSampler.DO_MULTIPART_POST">false</boolProp>
  <boolProp name="HTTPSampler.BROWSER_COMPATIBLE_MULTIPART">false</boolPro
p>

  <stringProp name="HTTPSampler.implementation">HttpClient4</stringProp>
</HTTPSamplerProxy>
<hashTree/>

<!-- 3 Шаг: Нажать кнопку "Войти" (имитация через GET-запрос с параметром
action) -->
  <HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProx
y" testname="03 – Нажатие кнопки 'Войти' (имитация)" enabled="true">
    <boolProp name="HTTPSampler.postBodyRaw">false</boolProp>

```



```
<elementProp name="HTTPSampler.Arguments" elementType="Arguments" guicla
ss="HTTPArgumentsPanel" testclass="Arguments" testname="Параметры запроса" enabled
="true">
```

```
  <collectionProp name="Arguments.arguments">
    <elementProp name="action" elementType="HTTPArgument">
      <boolProp name="HTTPArgument.always_encode">false</boolProp>
      <stringProp name="Argument.value">login</stringProp>
      <stringProp name="Argument.metadata">=</stringProp>
      <boolProp name="HTTPArgument.use_equals">true</boolProp>
      <stringProp name="Argument.name">action</stringProp>
    </elementProp>
    <elementProp name="login" elementType="HTTPArgument">
      <boolProp name="HTTPArgument.always_encode">false</boolProp>
      <stringProp name="Argument.value">admin</stringProp>
      <stringProp name="Argument.metadata">=</stringProp>
      <boolProp name="HTTPArgument.use_equals">true</boolProp>
      <stringProp name="Argument.name">login</stringProp>
    </elementProp>
    <elementProp name="pass" elementType="HTTPArgument">
      <boolProp name="HTTPArgument.always_encode">false</boolProp>
      <stringProp name="Argument.value">0000</stringProp>
      <stringProp name="Argument.metadata">=</stringProp>
      <boolProp name="HTTPArgument.use_equals">true</boolProp>
      <stringProp name="Argument.name">pass</stringProp>
    </elementProp>
```

```

    </collectionProp>
  </elementProp>
  <stringProp name="HTTPSampler.domain"></stringProp>
  <stringProp name="HTTPSampler.port"></stringProp>
  <stringProp name="HTTPSampler.protocol"></stringProp>
  <stringProp name="HTTPSampler.contentEncoding"></stringProp>
  <stringProp name="HTTPSampler.path">/test.html</stringProp>
  <stringProp name="HTTPSampler.method">GET</stringProp>
  <boolProp name="HTTPSampler.follow_redirects">true</boolProp>
  <boolProp name="HTTPSampler.auto_redirects">false</boolProp>
  <boolProp name="HTTPSampler.use_keepalive">true</boolProp>
  <boolProp name="HTTPSampler.DO_MULTIPART_POST">false</boolProp>
  <boolProp name="HTTPSampler.BROWSER_COMPATIBLE_MULTIPART">false</boolPro
p>

  <stringProp name="HTTPSampler.implementation">HttpClient4</stringProp>
</HTTPSamplerProxy>
<hashTree>
  <!-- Проверка: результат авторизации – успех -->
  <ResponseAssertion guiclass="AssertionGui" testclass="ResponseAssertion"
testname="Проверка: сообщение об успешной авторизации" enabled="true">
    <collectionProp name="Asserion.test_strings">
      <stringProp name="0">Вы успешно авторизованы</stringProp>
    </collectionProp>
    <stringProp name="Assertion.custom_message">Сообщение об успешной авто
ризации не найдено</stringProp>

```

```

        <stringProp name="Assertion.test_field">Assertion.response_data</stringProp>
    </ResponseAssertion>
    <hashTree/>
</hashTree>

<!-- Просмотр результатов в виде дерева (для отладки) -->
<ResultCollector guiclass="ViewResultsFullVisualizer" testclass="ResultCollector"
testname="Просмотр результатов" enabled="true">
    <boolProp name="ResultCollector.error_logging">false</boolProp>
    <objProp>
        <name>saveConfig</name>
        <value class="SampleSaveConfiguration">
            <time>true</time>
            <latency>true</latency>
            <timestamp>true</timestamp>
            <success>true</success>
            <label>true</label>
            <code>true</code>
            <message>true</message>
            <threadName>true</threadName>
            <dataType>true</dataType>
            <encoding>false</encoding>
        </value>
    </objProp>
</ResultCollector>

```

```

    <assertions>true</assertions>
    <subresults>true</subresults>
    <responseData>false</responseData>
    <samplerData>false</samplerData>
    <xml>false</xml>
    <fieldNames>true</fieldNames>
    <responseHeaders>false</responseHeaders>
    <requestHeaders>false</requestHeaders>
    <responseDataOnError>false</responseDataOnError>
    <saveAssertionResultsFailureMessage>true</saveAssertionResultsFailureMessage>

    <assertionsResultsToSave>0</assertionsResultsToSave>
    <bytes>true</bytes>
    <sentBytes>true</sentBytes>
    <url>true</url>
    <threadCounts>true</threadCounts>
    <idleTime>true</idleTime>
    <connectTime>true</connectTime>
  </value>
</objProp>
<stringProp name="filename"></stringProp>
</ResultCollector>
<hashTree/>

```

<!-- Сводный отчёт (Aggregate Report) -->

```
<ResultCollector guiclass="StatVisualizer" testclass="ResultCollector" tes
tname="Сводный отчёт" enabled="true">
  <boolProp name="ResultCollector.error_logging">false</boolProp>
  <objProp>
    <name>saveConfig</name>
    <value class="SampleSaveConfiguration">
      <time>true</time>
      <latency>true</latency>
      <timestamp>true</timestamp>
      <success>true</success>
      <label>true</label>
      <code>true</code>
      <message>true</message>
      <threadName>true</threadName>
      <dataType>true</dataType>
      <encoding>false</encoding>
      <assertions>true</assertions>
      <subresults>true</subresults>
      <responseData>false</responseData>
      <samplerData>false</samplerData>
      <xml>false</xml>
      <fieldNames>true</fieldNames>
      <responseHeaders>false</responseHeaders>
      <requestHeaders>false</requestHeaders>
      <responseDataOnError>false</responseDataOnError>
```

```

    <saveAssertionResultsFailureMessage>true</saveAssertionResultsFailure
eMessage>

    <assertionsResultsToSave>0</assertionsResultsToSave>
    <bytes>true</bytes>
    <sentBytes>true</sentBytes>
    <url>true</url>
    <threadCounts>true</threadCounts>
    <idleTime>true</idleTime>
    <connectTime>true</connectTime>
  </value>
</objProp>
  <stringProp name="filename"></stringProp>
</ResultCollector>
<hashTree/>

<!-- График результатов (Graph Results) -->
<ResultCollector guiclass="GraphVisualizer" testclass="ResultCollector" te
stname="График результатов" enabled="true">
  <boolProp name="ResultCollector.error_logging">false</boolProp>
  <objProp>
    <name>saveConfig</name>
    <value class="SampleSaveConfiguration">
      <time>true</time>
      <latency>true</latency>
      <timestamp>true</timestamp>
    </value>
  </objProp>
</ResultCollector>

```

```
<success>true</success>
<label>true</label>
<code>true</code>
<message>true</message>
<threadName>true</threadName>
<dataType>true</dataType>
<encoding>>false</encoding>
<assertions>true</assertions>
<subresults>true</subresults>
<responseData>>false</responseData>
<samplerData>>false</samplerData>
<xml>>false</xml>
<fieldNames>true</fieldNames>
<responseHeaders>>false</responseHeaders>
<requestHeaders>>false</requestHeaders>
<responseDataOnError>>false</responseDataOnError>
<saveAssertionResultsFailureMessage>true</saveAssertionResultsFailur
eMessage>
<assertionsResultsToSave>0</assertionsResultsToSave>
<bytes>true</bytes>
<sentBytes>true</sentBytes>
<url>true</url>
<threadCounts>true</threadCounts>
<idleTime>true</idleTime>
<connectTime>true</connectTime>
```

```
        </value>
      </objProp>
      <stringProp name="filename"></stringProp>
    </ResultCollector>
    <hashTree/>
  </hashTree>
</hashTree>
</jmeterTestPlan>
```

Настройки теста

Параметр	Значение	Описание
Пользователи	50	Количество параллельных виртуальных пользователей
Циклы	10	Количество повторений сценария каждым пользователем

Параметр	Значение	Описание
Разогрев (Ramp-Up)	5 сек	Время, за которое все 50 пользователей будут запущены
Задержка между шагами	1 сек	Пауза между действиями одного пользователя
Учётные данные	admin / 0000	Единственная пара, принимаемая формой



Как использовать

1. Сохраните приведённый выше XML-код в файл `example1.jmx`
2. Запустите **JMeter** и откройте этот файл через `File → Open`
3. Запустите тест кнопкой  (зелёная стрелка)
4. Анализируйте результаты во вкладках:

- *Сводный отчёт* — общая статистика (среднее время, пропускная способность, ошибки)
- *График результатов* — визуализация времени отклика
- *Просмотр результатов* — детали каждого запроса (для отладки)

Важное примечание

Страница <https://somovstudio.github.io/test.html> **не отправляет данные на сервер** — вся логика авторизации выполняется на стороне клиента (JavaScript) ¹. Поэтому в скрипте используются **GET-запросы с параметрами** для имитации действий пользователя. Фактическая нагрузка приходится на загрузку самой страницы, а не на обработку данных сервером.

Для тестирования реальных серверных форм потребуется заменить GET на POST и настроить соответствующие параметры.



20 web pages

🌀 Thought for 2 seconds >

Пожалуйста! Рад, что помогло 😊

Если понадобится подкорректировать нагрузку (изменить число пользователей, время разогрева, добавить разные логины/пароли или переделать под реальный POST-запрос) — просто напишите, я с удовольствием доработаю скрипт.

Удачи в тестировании! 🚀

а вовремя этого теста выполняются нажатия на кнопки и вводе данных в поля ?

🌀 Thought for 3 seconds >

Отличный вопрос! Короткий ответ: **нет, в этом тесте не выполняются реальные нажатия на кнопки и ввод в поля** в том смысле, как это делает браузер.

Давайте разберёмся подробнее.

Что делает наш скрипт

JMeter — это инструмент нагрузочного тестирования на уровне протоколов (HTTP/HTTPS). Он отправляет и получает **сырые HTTP-запросы** и анализирует ответы. Он **не рендерит страницу**, не выполняет **JavaScript** и не взаимодействует с **DOM-элементами** (поля ввода, кнопки).

В нашем скрипте мы **имитируем** действия пользователя с помощью GET-запросов с параметрами:

- Шаг 1 – просто загружаем страницу /test.html .
- Шаг 2 – отправляем GET-запрос на ту же страницу с параметрами
?login=admin&pass=0000 (это имитация заполнения полей).
- Шаг 3 – отправляем GET-запрос с параметрами
?action=login&login=admin&pass=0000 (имитация нажатия кнопки «Войти»).

Никакого реального клика или ввода текста не происходит — мы просто передаём данные в URL, как если бы пользователь вручную набрал их в адресной строке.

Почему так, а не иначе?

Потому что тестируемая страница (<https://somovstudio.github.io/test.html>) **не отправляет данные на сервер** — вся логика проверки логина и пароля написана на JavaScript и выполняется в браузере пользователя. При нажатии на кнопку «Войти» данные **не уходят** на сервер, поэтому JMeter не может «поймать» POST-запрос для нагрузочного тестирования.

Единственное, что можно проверить с помощью JMeter — это **скорость загрузки самой страницы** и её статических ресурсов (если бы они были). Именно это мы и делаем: все наши запросы просто загружают одну и ту же HTML-страницу с разными параметрами, и мы проверяем, что в ответе присутствует нужный текст («Вы успешно авторизованы» или заголовок «Авторизация»).

Как сделать реальное UI-тестирование с нажатиями?

Если вам нужно протестировать **именно взаимодействие с элементами интерфейса** (клики, ввод, проверка появления сообщений), то для этого существуют другие инструменты:

- **Selenium WebDriver** — автоматизация браузера, которая по-настоящему открывает страницу, вводит текст, кликает и проверяет результат.
- **JMeter + WebDriver Sampler** (плагин) — позволяет вставлять Selenium-код внутрь JMeter, но это уже смешанный подход, больше подходящий для функционального, а не нагрузочного тестирования.

Для нагрузочного тестирования UI обычно используют комбинацию: JMeter для HTTP-нагрузки (если есть серверная логика) и Selenium для проверки корректности отображения на небольшом количестве потоков.

Что же мы тестируем в нашем примере?

Мы тестируем способность сервера (GitHub Pages) отдавать страницу под нагрузкой. Поскольку страница статическая, нагрузка невелика, но скрипт демонстрирует структуру теста: таймеры, проверки ответов, отчёты. Он показывает, как можно организовать имитацию пользовательских действий через параметры запросов, если серверная часть их обрабатывает.

Итог

-  Реальных нажатий и ввода нет — только HTTP-запросы.
-  Для UI-взаимодействия нужен Selenium.
-  Наш пример полезен как учебный и для тестирования статики или серверной логики, где данные передаются через GET/POST.

Если вам нужен тест с реальными кликами — дайте знать, я помогу настроить Selenium или адаптировать JMeter под вашу задачу!